Programarea calculatoarelor

FMI

Secția Calculatoare și tehnologia informației, anul I

Cursul 6 / 10.11.2024

Programa cursului

1. Introducere
- Algoritmi
- Limbaje de programare.
- Introducere în limbajul C. Structura unui program C

2. Fundamentele limbajului C
- Etapele realizării unui program C
- Tipuri de date fundamentale. Variabile. Constante. Operatori. Expresii. Conversii
- Tipuri derivate de date: tablouri, șiruri de caractere, structuri, uniuni, câmpuri de biți, enumerări, pointeri
- Instrucțiuni de control
- Directive de preprocesare. Macrodefiniții
- Funcții de citire/scriere

3. Fișiere text
- **Funcții specifice de manipulare**

4. Funcții (1)
- **Declarare și definire. Apel.**
- **Metode de trasmitere a paramerilor.**
- **Pointeri la funcții**

5. Tablouri și pointeri
- Legătura dintre tablouri și pointeri
- Aritmetica pointerilor
- Alocarea dinamică a memoriei
- Clase de memorare

6. Șiruri de caractere
- Funcții specifice de manipulare

7. Fișiere binare
- Funcții specifice de manipulare

8. Structuri de date complexe și autoreferite
- Definire și utilizare

9. Funcții (2)
- Funcții cu număr variabil de argumente
- Preluarea argumentelor funcției main din linia de comandă
- Programare generică

Cuprinsul cursului de azi

1. Fișiere text
2. Funcții

1. Fișiere text

A. Fișiere

1) Fișier. Definiție
2) Tipuri de fișiere
3) Lucrul cu fișiere

B. Fișiere text. Funcții specifice de manipulare

1) Fișiere

- **fișier = șir de octeți (colecție de date) memorat pe suport extern (magnetic, optic) și identificat printr-un nume**
- fișierele sunt entități ale sistemului de operare
- operațiile cu fișiere se realizează de către sistemul de operare, compilatorul de C traduce funcțiile de acces la fișiere în apeluri ale funcțiilor de sistem; alte limbaje de programare fac același lucru

Fișiere

- noțiunea de fișier este mai generală:

fișier = flux de date (stream) = transfer de informație binară (șir de octeți) de la o sursă spre o destinație:

- ✓ **citire**: flux de la tastatură (sursă) către memoria internă (destinație)
- ✓ **afișare**: flux de la memoria internă (sursă) către periferice (monitor, imprimantă)

Fluxuri care se deschid automat în program

- **stdin (standard input)** – flux de intrare (citire)
  - ✓ asociat implicit cu tastatura

- **stdout (standard output)** – flux de ieșire (afișare)
  - ✓ asociat implicit cu ecranul

- **stderr (standard error)** – flux de ieșire (afișare) pentru erori
  - ✓ asociat implicit cu ecranul

2) Tipuri de fișiere

1. **fișiere text:** fiecare octet este interpretat drept caracter cu codul ASCII dat de octetul respectiv
   - octeții (caractere ASCII) sunt organizați pe linii
   - un fișier text **poate** fi terminat printr-un caracter terminator de fișier (**EOF = CTRL-Z**)
     - ✓ caracterele terminatorii de linii : **CR + LF = '\r\n'** (carriage return și line feed)

       (\r are codul ASCII 13, \n are codul ASCII 10, dar se returnează doar \n)
     - ✓ Terminatorul nu este obligatoriu. Sfârşitul unui fişier disc poate fi detectat şi pe baza lungimii fişierului (număr de octeţi), memorată pe disc.

Exemplu: un fișier text în care scriem numărul întreg 123 ocupă trei octeți (codul ASCII pt 1, codul ASCII pt 2, codul ASCII pt 3)

Tipuri de fișiere

2. **fișiere binare:** octeții nu sunt organizați în nici un fel
   - nu există noțiunea de linie

Exemplu: un fișier binar în care scriem numărul întreg 123 ocupă 4 octeți (scrierea binară a lui 123 în baza 2 pentru un int)

3) Lucrul cu fișiere

- în biblioteca **stdio.h** sunt definite:
  - ✓ o structură **FILE** și
  - ✓ funcțiile necesare
    - ➢ deschiderii,
    - ➢ închiderii,
    - ➢ scrierii și
    - ➢ citirii din fișiere.

Lucrul cu fișiere

a) declararea unui **pointer la structura FILE** = **realizarea legăturii dintre nivelele logic (variabila fișier) și fizic (numele extern al fișierului):**

FILE * f;

- **Cererea de deschidere a unui fișier:**
  - ✓ fișierul a putut fi deschis:
    - ➢ pointer-ul la **FILE nu este NULL**
    - ➢ se prelucrează fișierul și se închide
  - ✓ fișierul nu a putut fi deschis:
    - ➢ pointer-ul la **FILE este NULL**
    - ➢ nu se poate continua cu prelucrarea
    - ➢ nu este necesară închiderea

main.c | untitled_2.c | untitled_3.c

```c
#include <stdio.h>
#include <errno.h>
#include <string.h>
#define CLOSE_FILE 1
void verify_file(FILE *f)
{
    if (f!=NULL)
    {
        printf("%p\n",f);
        if (CLOSE_FILE)
            fclose(f);
    }
    else
    {
        printf("Nu am putut deschide fisierul! Eroare %s\n",strerror(errno));
    }
}

int main()
{
    FILE *f1, *f2, *f3;
    f1=fopen("untitled_2.c", "r");
    verify_file(f1);
    f2=fopen("untitled_3.c", "r");
    verify_file(f2);
    f2=fopen("untitled_30.c", "r");
    verify_file(f2);
    printf("Numarul maxim de fisiere deschise: %d",FOPEN_MAX);

    return 0;
}
```

main.c | untitled_2.c | untitled_3.c

```c
#include <stdio.h>
#include <errno.h>
#include <string.h>
#define CLOSE_FILE 1
void verify_file(FILE *f)
{
    if (f!=NULL)
    {
        printf("%p\n",f);
        if (CLOSE_FILE)
            fclose(f);
    }
    else
    {
        printf("Nu am putut deschide fisierul! Eroare %s\n",strerror(errno));
    }
}

int main()
{
    FILE *f1, *f2, *f3;
    f1=fopen("untitled_2.c", "r");
    verify_file(f1);
    f2=fopen("untitled_3.c", "r");
    verify_file(f2);
    f2=fopen("untitled_30.c", "r");
    verify_file(f2);
    printf("Numarul maxim de fisiere deschise: %d",FOPEN_MAX);

    return 0;
}
```

```
0x560faa1e32a0
0x560faa1e32a0
Nu am putut deschide fisierul! Eroare No such file or directory
Numarul maxim de fisiere deschise: 16
```

Lucrul cu fișiere

b) deschiderea unui fișier = stabilirea unui flux către acel fișier.

Se realizează folosind **funcția fopen**:

FILE *fopen(char *nume_fisier, char *mod_acces)

- ✓ **nume_fisier** = numele fișierului
- ✓ **mod_acces** = șir de 1-3 caractere ce indică tipul de acces :
  - ➢ citire ”**r**”, scriere ”**w**”, adăugare la sfârșitul fișierului ”**a**”;
  - ➢ ”**+**” permite scrierea și citirea ”**r+**”,”**w+**”,”**a+**”;
  - ➢ **t** (implicit) sau **b**: specifică tipul de fișier (text sau binar).

- funcția **fopen** întoarce un pointer la o structura **FILE** sau în caz de eroare (fișierul nu se poate deschide) întoarce NULL.

Lucrul cu fișiere

Moduri de prelucrare a fișierelor text

Mod	Descriere
r	Deschiderea fișierului pentru citire. Fișierul trebuie să existe!
w	Crearea unui fișier pentru scriere. Dacă fișierul există, conținutul acestuia este șters în totalitate!
a	Deshiderea sau crearea (dacă nu există) unui fișier pentru adăugarea de conținut numai la sfârșitul acestuia
r+	Deschiderea unui fișier pentru actualizarea conținutului (citire și scriere). Fișierul trebuie să existe!
w+	Deschiderea unui fișier pentru actualizarea conținutului (citire și scriere). Dacă fișierul există, conținutul acestuia este șters în totalitate!
a+	Deschiderea unui fișier pentru citirea conținutului și adăugarea de conținut numai la sfârșitul acestuia. Dacă fișierul nu există, acesta este creat.

Pentru lucrul cu fisierul in mod binar avem variante similare:
“rb”, “wb”, “ab”, “r+b”, “w+b”, “a+b”.

Lucrul cu fișiere

Ce afișează programul?

main.c | untitled_1.txt | untitled_3.c

```c
#include <stdio.h>
#include <stdlib.h>
//#include <string.h>

int main()
{
    FILE *g=fopen("untitled_1.txt","r+");
    char sir[10];

    if (g==NULL)
        printf("Eroare");

    fgets(sir,10,g);
    printf("%s",sir);

    return 0;
}
```

Lucrul cu fișiere

main.c | untitled_1.txt | untitled_3.c

```c
#include <stdio.h>
#include <stdlib.h>
//#include <string.h>

int main()
{
    FILE *g=fopen("untitled_1.txt","r+");
    char sir[10];

    if (g==NULL)
        printf("Eroare");

    fgets(sir,10,g);
    printf("%s",sir);

    return 0;
}
```

main.c | untitled_1.txt | untitled_3.c

```
1  abecedar
```

```
abecedar

...Program finished with exit code
Press ENTER to exit console.
```

Citește un șir de caractere din fișier și îl afișează pe ecran.

Lucrul cu fișiere

Ce afișează programul?

```c
#include <stdio.h>
#include <stdlib.h>

int main()
{
    FILE *g = fopen("fisier.txt", "r+");
    char c;

    if (g == NULL)
        printf("\nNu s-a putut deschide!\n");

    c = fgetc(g);
    printf("%c", c);

    fputc('z', g);

    if (fclose(g) != 0)
        printf("\nProbleme la inchiderea fisierului!\n");

    return 0;
}
```

Lucrul cu fișiere

```c
#include <stdio.h>
#include <stdlib.h>

int main()
{
    FILE *g = fopen("fisier.txt", "r+");
    char c;

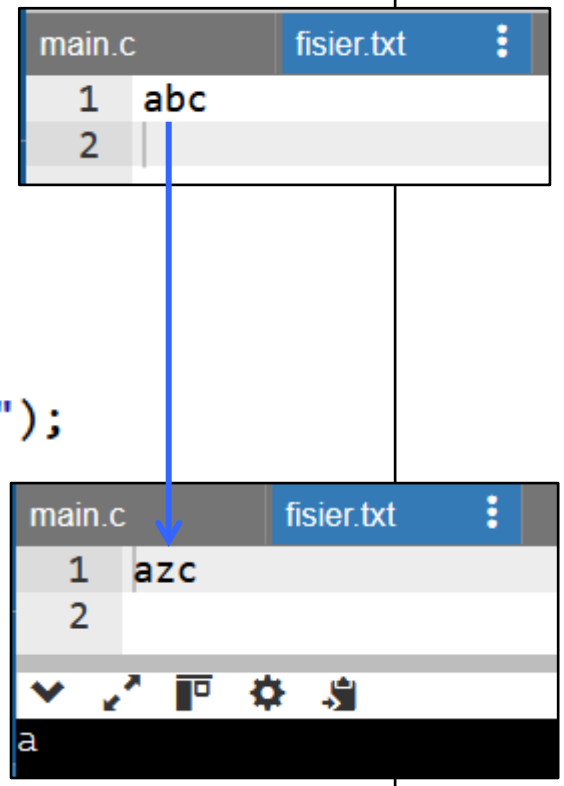
    if (g == NULL)
        printf("\nNu s-a putut deschide!\n");

    c = fgetc(g);
    printf("%c", c);

    fputc('z', g);

    if (fclose(g) != 0)
        printf("\nProbleme la inchiderea fisierului!\n");

    return 0;
}
```



Rezultat: scrie primul caracter pe ecran, suprascriere al doilea caracter în fișier

Lucrul cu fișiere

Ce afișează programul?

```c
#include <stdio.h>
#include <stdlib.h>

int main()
{
    FILE *g = fopen("fisier.txt", "r+");
    char c;

    if (g == NULL)
        printf("\nNu s-a putut deschide!\n");

    c = fgetc(g);
    printf("%c", c);
    fclose(g);

    g = fopen("fisier.txt", "r+");
    fputc('X', g);

    if (fclose(g) != 0)
        printf("\nProbleme la inchiderea fisierului!\n");

    return 0;
}
```

Lucrul cu fișiere

```c
#include <stdio.h>
#include <stdlib.h>

int main()
{
    FILE *g = fopen("fisier.txt", "r+");
    char c;

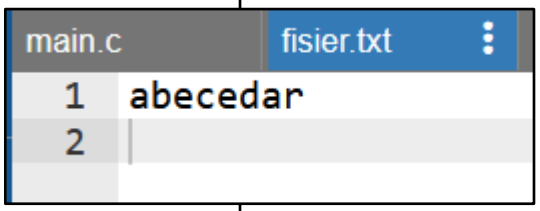
    if (g == NULL)
        printf("\nNu s-a putut deschide!\n");

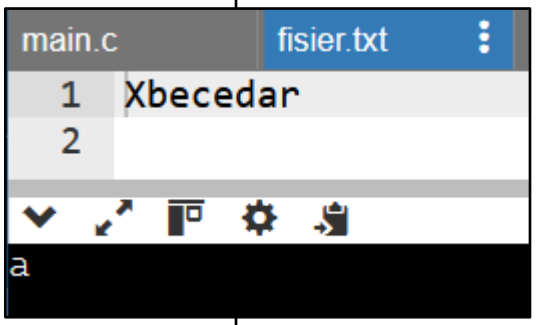
    c = fgetc(g);
    printf("%c", c);
    fclose(g);

    g = fopen("fisier.txt", "r+");
    fputc('X', g);

    if (fclose(g) != 0)
        printf("\nProbleme la inchiderea fisierului!\n");

    return 0;
}
```





La deschiderea/ redeschiderea fișierului cursorul se poziționează înaintea primului caracter.

Lucrul cu fișiere

c) Redeschiderea fișierului implică **resetarea cursorului în punctul de început** al fișierului!

- Adăugând **+** după modificatorul de acces (e.g. w+, r+, a+), fișierul se deschide cu **permisiuni de citire/scriere**.
- Cu toate acestea:
  a) După ce s-a citit din fișier, va trebui apelată o funcție de poziționare în fișier (**fseek, fsetpos, rewind**)
  b) După scriere, va trebui apelată **fflush()** sau o funcție de poziționare în fișier, înainte de a citi.

=> Ce facem dacă dorim scrierea la poziția curentă a cursorului?

Poziționarea cursorului

```c
#include <stdio.h>
#include <stdlib.h>

int main()
{
    FILE *g = fopen("fisier.txt", "r+");

    if (g == NULL)
        printf("\nNu s-a putut deschide!\n");
    else
    {
        printf("%d\n", ftell(g));

        char c = fgetc(g);
        printf("%c\n", c);
        printf("%d\n", ftell(g));

        fseek(g, 3, SEEK_CUR);
        fputc('z', g);
        fflush(g);
        printf("%d\n", ftell(g));

        if (fclose(g) != 0)
            printf("\nProbleme la inchidere!\n");
    }
    return 0;
}
```

Funcția ftell returnează pozitia curentă a cursorului în cadrul fișierului

Poziționarea cursorului

```c
#include <stdio.h>
#include <stdlib.h>

int main()
{
    FILE *g = fopen("fisier.txt", "r+");

    if (g == NULL)
        printf("\nNu s-a putut deschide!\n");
    else
    {
        printf("%d\n", ftell(g));

        char c = fgetc(g);
        printf("%c\n", c);
        printf("%d\n", ftell(g));

        fseek(g, 3, SEEK_CUR);
        fputc('z', g);
        fflush(g);
        printf("%d\n", ftell(g));

        if (fclose(g) != 0)
            printf("\nProbleme la inchidere!\n");
    }
    return 0;
}
```

Funcția fseek poziționează "cursorul" (virtual) de citire/scriere oriunde în interiorul fișierului
(aici la 3 pozitii dupa pozitia curenta, care este a doua in g, deci cu numarul de ordine 1)

Poziționarea cursorului

```c
#include <stdio.h>
#include <stdlib.h>

int main()
{
    FILE *g = fopen("fisier.txt", "r+");

    if (g == NULL)
        printf("\nNu s-a putut deschide!\n");
    else
    {
        printf("%d\n", ftell(g));

        char c = fgetc(g);
        printf("%c\n", c);
        printf("%d\n", ftell(g));

        fseek(g, 3, SEEK_CUR);
        fputc('z', g);
        fflush(g);
        printf("%d\n", ftell(g));

        if (fclose(g) != 0)
            printf("\nProbleme la inchidere!\n");
    }
    return 0;
}
```

Funcția fflush forțează scrierea datelor în fișier fără închiderea acestuia

Poziționarea cursorului

```c
#include <stdio.h>
#include <stdlib.h>

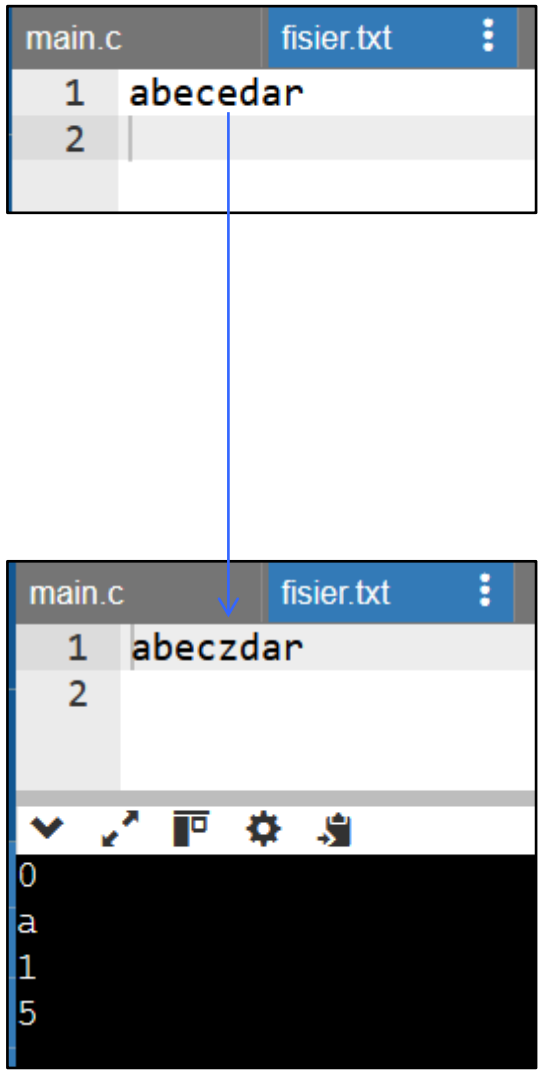
int main()
{
    FILE *g = fopen("fisier.txt", "r+");

    if (g == NULL)
        printf("\nNu s-a putut deschide!\n");
    else
    {
        printf("%d\n", ftell(g));

        char c = fgetc(g);
        printf("%c\n", c);
        printf("%d\n", ftell(g));

        fseek(g, 3, SEEK_CUR);
        fputc('z', g);
        fflush(g);
        printf("%d\n", ftell(g));

        if (fclose(g) != 0)
            printf("\nProbleme la inchidere!\n");
    }
    return 0;
}
```



Poziționarea cursorului

main.c

```c
#include <stdio.h>
#include <stdlib.h>

int main()
{
    FILE *g = fopen ("fisier.txt", "r+");
    char c;
    if (g == NULL)
        printf("\n Nu s-a putut deschide! \n");

    c = fgetc(g);
    printf("%c",c);

    fseek(g,0,SEEK_CUR);

    fputc('z',g);
    if (fclose(g) != 0)
        printf("\n Probleme la inchiderea fisierului! \n");

    return 0;
}
```

Ce afișează programul?

```
 Nu s-a putut deschide!


...Program finished with exit code 0
Press ENTER to exit console.
```

Poziționarea cursorului

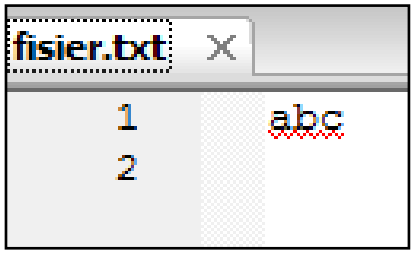
main.c | fisier.txt

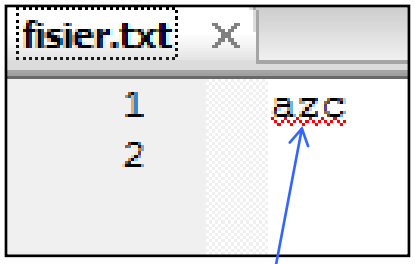
```
1  #include <stdio.h>
2  #include <stdlib.h>
3
4  int main()
5  {
6      FILE *g = fopen ("fisier.txt", "r+");
7      char c;
8      if (g == NULL)
9          printf("\n Nu s-a putut deschide! \n");
10
11     c = fgetc(g);
12     printf("%c",c);
13
14     fseek(g,0,SEEK_CUR);
15
16     fputc('z',g);
17     if (fclose(g) != 0)
18         printf("\n Probleme la inchiderea fisierului! \n");
19
20     return 0;
21 }
22
```

input

```
a

...Program finished with exit code 0
Press ENTER to exit console.
```





Schimbă caracterul **de la poziția curentă**!

Lucrul cu fișiere

d) închiderea unui fișier = închiderea unui flux către acel fișier.

Se realizează folosind **funcția fclose**:

int *fclose(FILE *f)

f = pointer la FILE care realizează legătura cu fișierul pe care vreau să-l închid

- **fclose** întoarce:
  - ✓ **0** dacă închiderea s-a efectuat cu succes și
  - ✓ **EOF** în caz de eroare.

- Toate fișierele în care s-a **scris** trebuie **închise**.
- Dacă se realizează doar citire, fisierul nu trebuie închis.
- Tastatura şi imprimanta sunt considerate fişiere text. Ele nu trebuie deschise şi închise.

Lucrul cu fișiere

main.c | fisier.txt

```c
#include <stdio.h>

int main()
{
    int a;

    FILE *g = fopen("fisier.txt", "w");

    if (g == NULL)
        printf("Nu s-a putut deschide fisierul.\n");

    fputc ('A', g);

    a=fclose(g);
    printf("%d \n", a);

    return 0;
}
```

Ce afișează programul?

main.c | fisier.txt

```
1  abecedar
```

main.c | fisier.txt

```
1  A
```

```
0
```

Funcția fclose întoarce 0 dacă închiderea s-a efectuat cu succes

Lucrul cu fișiere

e) detectarea sfârșitului de fișier - se poate realiza și folosind funcția **feof (*find end of file*):**

int feof(FILE *f)

f = pointer la FILE corespunzătoare fișierului pe care îl prelucrăm

- funcția **feof** returnează:
  - ✓ **0** dacă nu s-a ajuns la sfârșitul fișierului la ultima operație de citire
  - ✓ **o valoare nenulă** dacă s-a ajuns la sfârșitul fișierului

Lucrul cu fișiere

main.c | fisier.txt

```c
#include <stdio.h>
#include <stdlib.h>
int main()
{
    char c;

    FILE *g = fopen("fisier.txt", "r+");

    if (g == NULL)
        printf("Eroare!");

    while(!feof(g))
    {
        c = fgetc(g);
        printf("%c(%d) ", c, c);
    }

    return 0;
}
```

fisier.txt

```
1  abc
2
```

Detectarea sfârșitului de fișier

Lucrul cu fișiere

main.c | fisier.txt

```
1  #include <stdio.h>
2  #include <stdlib.h>
3  int main()
4  {
5      char c;
6
7      FILE *g = fopen("fisier.txt", "r+");
8
9      if (g == NULL)
10         printf("Eroare!");
11
12     while(!feof(g))
13     {
14         c = fgetc(g);
15         printf("%c(%d) ", c, c);
16     }
17
18     return 0;
19 }
```

```
a(97) b(98) c(99)
(10) ?(-1)
```

fisier.txt

```
1  abc
2
```

ASCII code of '\n' is 10, a non-zero value for feof

Fișiere text.
Funcții specifice de manipulare

1) Funcții de citire/ scriere la nivel de caracter
2) Funcții de citire/ scriere la nivel de linie
3) Funcții de citire/ scriere cu format
4) Funcții de poziționare într-un fișier
5) Alte funcții

1) Funcții de citire/scriere la nivel de caracter

- **int fgetc(FILE *f)**
  - ✓returneaza codul ASCII al caracterului citit din fișierul **f**
  - ✓ întoarce EOF (= -1) dacă s-a ajuns la finalul fișierului sau a avut loc o eroare la citire
- **int fputc(int c, FILE *f)**
  - ✓scrie caracterul cu codul ASCII **c** în fișierul **f**
  - ✓ întoarce:
    - EOF (= -1) în caz de eroare
    - codul ASCII al caracterului scris în caz de succes

Funcții de citire/scriere la nivel de caracter

```c
#include <stdio.h>
#include <stdlib.h>

int main()
{
    char nume[15] = "fisier_copy.txt";
    FILE *f = fopen("fisier.txt", "r");
    FILE *g = fopen(nume, "w");
    char c;

    if (f == NULL)
        printf("\nNu s-a putut deschide!\n");

    while((c = fgetc(f)) != EOF)
        fputc(c, g);


    if ((fclose(f) != 0) || (fclose(g) != 0))
            printf("\nProbleme la inchidere!\n");

    return 0;
}
```

Funcții de citire/scriere la nivel de caracter

```c
#include <stdio.h>
#include <stdlib.h>

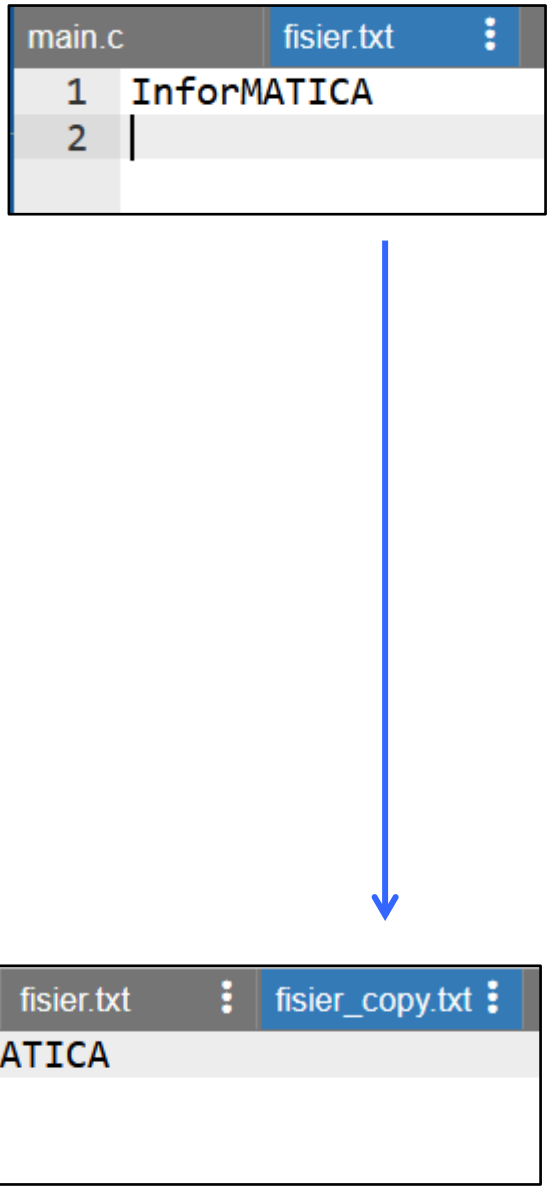
int main()
{
    char nume[15] = "fisier_copy.txt";
    FILE *f = fopen("fisier.txt", "r");
    FILE *g = fopen(nume, "w");
    char c;

    if (f == NULL)
        printf("\nNu s-a putut deschide!\n");

    while((c = fgetc(f)) != EOF)
        fputc(c, g);


    if ((fclose(f) != 0) || (fclose(g) != 0))
        printf("\nProbleme la inchidere!\n");

    return 0;
}
```



2) Funcții de citire/scriere la nivel de linie

- **char* fgets(char *sir, int m, FILE *f)**
  - ✓citește maxim **m-1** caractere sau până la '**\n**' și pune șirul de caractere în **sir** (adaugă la sfârșit '**\0**')
  - ✓ returnează:
    - adresa șirului citit sau
    - **NULL** în caz de eroare
- **int fputs(char *sir, FILE *f)**
  - ✓scrie șirul **sir** în fișierul **f**, fără a pune '**\n**' la sfârșit
  - ✓întoarce:
    - numarul de caractere scrise sau
    - **EOF** în caz de eroare

Funcții de citire/scriere la nivel de linie

main.c | fisier.txt

```c
#include <stdio.h>
#include <stdlib.h>

int main()
{

   FILE *g = fopen ("fisier.txt", "r+");
   char sir[30];


   if (g ==NULL)
      printf("\nNu s-a putut deschide fisierul!");

    fgets(sir,20,g);
    printf("%s",sir);

    if (fclose(g)!=0)
       printf("\nProbleme la inchiderea fisierului");

   return 0;
}
```

Ce afișează programul?

Funcții de citire/scriere la nivel de linie

main.c | fisier.txt

```
1  abecedarABECEDARcalendarCALENDAR
```

```
abecedarABECEDARcal
```

main.c | fisier.txt

```c
#include <stdio.h>
#include <stdlib.h>

int main()
{

   FILE *g = fopen ("fisier.txt", "r+");
   char sir[30];


   if (g ==NULL)
      printf("\nNu s-a putut deschide fisierul!");

    fgets(sir,20,g);
    printf("%s",sir);

    if (fclose(g)!=0)
       printf("\nProbleme la inchiderea fisierului");

   return 0;
}
```

3) Funcții de citire/scriere cu format

- **int fscanf(FILE *f, char *format)**
  - ✓ citește din fisierul **f** folosind un format (analog cu **scanf**)

- **int fprintf(FILE *f, char *format)**
  - ✓ scrie în fișierul **f** folosind un format (analog cu **printf**)

Funcții de citire/scriere cu format

main.c numar.txt

```c
#include <stdio.h>
#include <stdlib.h>

int main()
{
    FILE *f = fopen("numar.txt", "w");
    int x;

    if (f == NULL)
        fprintf(stdout, "\nNu s-a putut deschide!\n");

    fscanf(stdin, "%d", &x);
    fprintf(f, "%d \n", x);

    fclose(f);

    f = fopen("numar.txt", "r");

    fscanf(f, "%d", &x);
    fprintf(stdout, "numar citit din fisier: %d \n", x);

    if (fclose(f) != 0)
        printf("\nProbleme la inchidere!\n");

    return 0;
}
```

Funcții de citire/scriere cu format

main.c | numar.txt

```c
#include <stdio.h>
#include <stdlib.h>

int main()
{
    FILE *f = fopen("numar.txt", "w");
    int x;

    if (f == NULL)
        fprintf(stdout, "\nNu s-a putut deschide!\n");

    fscanf(stdin, "%d", &x);
    fprintf(f, "%d \n", x);

    fclose(f);

    f = fopen("numar.txt", "r");

    fscanf(f, "%d", &x);
    fprintf(stdout, "numar citit din fisier: %d \n", x);

    if (fclose(f) != 0)
        printf("\nProbleme la inchidere!\n");

    return 0;
}
```

main.c | numar.txt

```
1 1234
2
```

```
1234
numar citit din fisier: 1234
```

Funcții de citire/scriere cu format

```c
#include<stdio.h>
#include<stdlib.h>

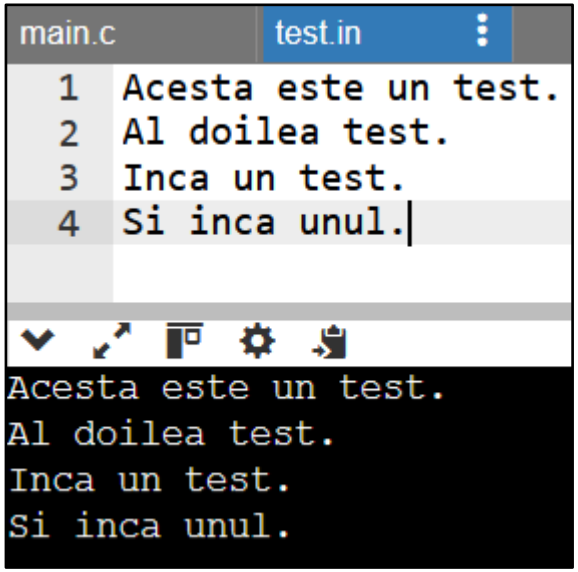
int main()
{
    FILE *fp;
    char s[80];

    if ( (fp = fopen("test.in","r")) == NULL )
    {
        printf("Nu se poate deschide fisierul!\n");
        exit (1);
    }

    while (fgets(s,80,fp) != NULL)
        printf ("%s", s);

    fclose (fp);

    return 0;
}
```



Citire linie cu linie dintr-un fişier şi afişare pe ecran

Funcții de citire/scriere cu format

main.c

```c
#include<stdio.h>
#include<stdlib.h>

int main()
{
    FILE *f;
    int x;

    f = fopen("numere.txt", "w");
    for(x = 1; x <= 10; x++)
        fprintf(f, "%d ", 2*x-1);

    fclose(f);

    f = fopen("numere.txt", "r");
    if (f == NULL)
    {
        printf("Nu se poate deschide fisierul!\n");
        exit (1);
    }
    while (fscanf(f,"%d", &x) == 1)
        printf("%d ", x);

    fclose(f);
    return 0;
}
```

Funcții de citire/scriere cu format

Creare fișier, scriere în fișier, citire date din fișier

main.c

```c
#include<stdio.h>
#include<stdlib.h>

int main()
{
    FILE *f;
    int x;

    f = fopen("numere.txt", "w");
    for(x = 1; x <= 10; x++)
        fprintf(f, "%d ", 2*x-1);

    fclose(f);

    f = fopen("numere.txt", "r");
    if (f == NULL)
    {
        printf("Nu se poate deschide fisierul!\n");
        exit (1);
    }
    while (fscanf(f,"%d", &x) == 1)
        printf("%d ", x);

    fclose(f);
    return 0;
}
```

main.c | numere.txt

```
1 3 5 7 9 11 13 15 17 19
```

```
1 3 5 7 9 11 13 15 17 19
```

```c
while(!feof(f))
{
    fscanf(f,"%d", &x);
    printf("%d ", x);
}
```

4) Funcții de poziționare într-un fișier

- în C ne putem poziționa pe un anumit octet din fișier
- funcțiile care permit poziționarea (cele mai importante):
- **long int ftell(FILE *f)**
  - ✓ întoarce numărul octetului curent față de începutul fișierului;
  - ✓ (dimensiunea maximă a unui fișier în C este de 2^{31}-1 octeți ~ 2GB)
- **int fseek(FILE *f, int nr_octeti, int origine)**
  - ✓ mută pointerul de fișier **f** pe octetul numărul **nr_octeti** in raport cu **origine**
  - ✓ origine – 3 valori posibile:
    - **SEEK_SET (= 0) - început de fișier**
    - **SEEK_CUR (=1) – poziția curentă**
    - **SEEK_END (=2) – sfârșit de fișier**

Funcții de poziționare într-un fișier

Aflarea dimensiunii unui fișier

```c
#include<stdio.h>
#include<stdlib.h>

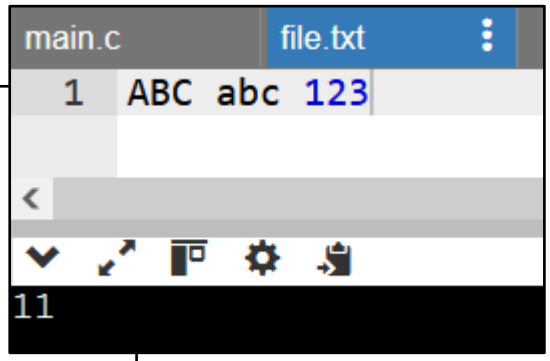
int main()
{
    FILE *g = fopen ("file.txt", "r+");
    long int nr_octeti;

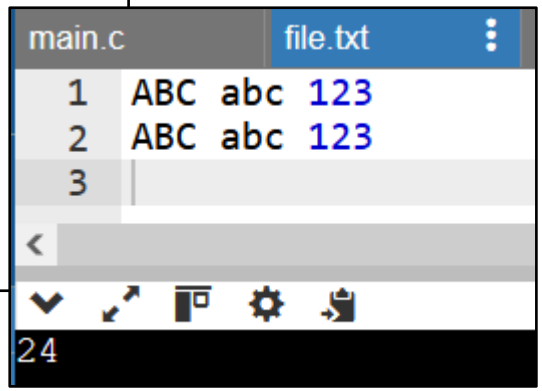
    if (g == NULL)
        printf("Nu s-a putut deschide fisierul!\n");

    fseek(g, 0, SEEK_END);
    nr_octeti = ftell(g);
    printf("%ld", nr_octeti);

    if (fclose(g) != 0)
        printf("\nProbleme la inchidere!\n");

    return 0;
}
```





5) Alte funcții pentru lucrul cu fișiere

- **void rewind (FILE *f)**
  - ✓ repoziționarea pointerului asociat fișierului la începutul său.
- **int remove(char * nume_fisier);**
  - ✓ șterge fișierul cu numele = **nume_fișier**
  - ✓ întoarce: **0** în caz de succes, **1** în caz de eroare
- **int rename(char *nume_vechi, char *nume_nou);**
  - ✓ redenumește fișierul cu numele = **nume_vechi** cu **nume_nou**
  - ✓ întoarce **0** în caz de succes, **1** în caz de eroare

Cuprinsul cursului de azi

1. Fișiere text
2. **Funcții**

Funcții

1. **Declarare și definire. Apel.**
2. **Metode de trasmitere a paramerilor.**
3. **Pointeri la funcții**

Funcții (recapitulare)

- permit modularizarea programelor
  - ✓ variabilele declarate în interiorul funcțiilor = variabile locale (vizibile doar în interior)
- parametrii funcțiilor
  - ✓ permit comunicarea informației între funcții
  - ✓ sunt variabile locale ale funcțiilor
- avantajele utilizării funcțiilor:
  - ✓ divizarea problemei în subprobleme
  - ✓ managementul dezvoltării programelor
  - ✓ utilizarea/reutilizarea funcțiilor scrise în alte programe
  - ✓ elimină duplicarea codului scris

Funcții

- o funcție = bloc de instrucțiuni care nu se poate executa de sine stătător, ci trebuie apelat

Sintaxa:

antetul funcției (declarare)

tip_returnat nume_functie (lista parametrilor formali)

{ variabile locale

instructiuni;

return expresie;

}

corpul funcției (definire)

- lista de parametri formali poate fi reprezentată de:
  - ✓ nici un parametru:
    - ➢ **tip_returnat nume_functie ()**
    - ➢ **tip_returnat nume_functie (void)**
  - ✓ unul sau mai mulți parametri separați prin virgulă

Valoarea returnată de o funcție

- două categorii de funcții:
  - ✓care **returnează o valoare**: prin utilizarea instrucțiunii
    return expresie;
  - ✓care **nu returnează o valoare**: prin instrucțiunea **return;** (tipul returnat este **void**)

- returnarea valorii
  - ✓ o funcție poate returna:
    - ➢ orice tip standard (**void, char, int, float, double**) sau
    - ➢ definit de utilizator (**structuri, uniuni, enumerari, typedef**)
  - ✓declarațiile și instrucțiunile din funcții sunt executate până se întâlnește
    - ➢ instrucțiunea **return** sau
    - ➢ acolada închisă **}** - execuția atinge finalul funcției

Valoarea returnată de o funcție

```
double f(double t)
{
    return t-1.5;
}
```

definire de funcție

```
float g(int);
```

declarație de funcție (prototip)

```
int main()
{
    float a=11.5;
    printf("%f\n",f(a));
    printf("%f\n",g(a));
    return 0;
}
```

```
float g(int z)
{
    return z+2.0;
}
```

definire de funcție

Prototipul și argumentele funcțiilor

- **prototipul** unei funcții (declararea ei) constă în specificarea antetului urmat de caracterul **;**
  - ✓ nu este necesară specificarea numelor parametrilor formali

    int adunare(int, int);
  - ✓ este necesară inserarea prototipului unei funcții înaintea altor funcții în care este invocată dacă definirea ei este localizată după definirea acelor funcții
- **parametrii** apar în definiții
- **argumentele** apar în apelurile de funcții
- corespondența între **parametrii formali** (din definiția funcției) și **actuali** (la apelul funcției) este **pozițională**
- regula de **conversie a argumentelor:** în cazul în care diferă, tipul fiecărui argument este convertit automat la tipul parametrului formal corespunzător (ca și în cazul unei simple atribuiri)

Conversia argumentelor funcțiilor

```
1  #include <stdio.h>
2
3  void f(char a)
4  {
5      printf("%d\n", a);
6  }
7  int main()
8  {
9      int a = 300;
10     float b = 305.7;
11     f(a);
12     f(b);
13
14     return 0;
15 }
```

char $\in$[0,256)
300 – 256 = 44
305 – 256 = 49

Rezultatul afișat:

44
49

Fișiere header cu extensia .h

- conțin **prototipuri de funcții**
- **bibliotecile standard**:
  - ✓ conțin prototipuri de funcții standard regăsite în fișierele *header* corespunzătoare
  - ✓ *exemple:* **stdio.h**, **stdlib.h**, **math.h**, **string.h**
  - ✓ se încarcă cu **#include <filename.h>**

- **biblioteci utilizator**
  - ✓ conțin prototipuri de funcții și macrouri
  - ✓ se pot salva ca fișiere cu extensia *.h*
  - ✓ *exemplu*: **filename.h**
  - ✓ se încarcă cu **#include “filename.h”**

Funcții

1. Declarare și definire. Apel.
2. **Metode de trasmitere a paramerilor.**
3. **Pointeri la funcții**

Transmiterea parametrilor către funcții

- utilizată la apelul funcțiilor

- **în limbajul C transmiterea parametrilor se poate face doar prin valoare (pass-by-value)**
  - o copie a argumentelor este trimisă funcției
  - modificările în interiorul funcției nu afectează argumentele originale

- în **limbajul C++** transmiterea parametrilor apelul se poate face și prin **referință (pass-by-reference)**
  - ✓ argumentele originale sunt trimise funcției
  - ✓ modificările în interiorul funcției afectează argumentele trimise

interchimbare.cpp

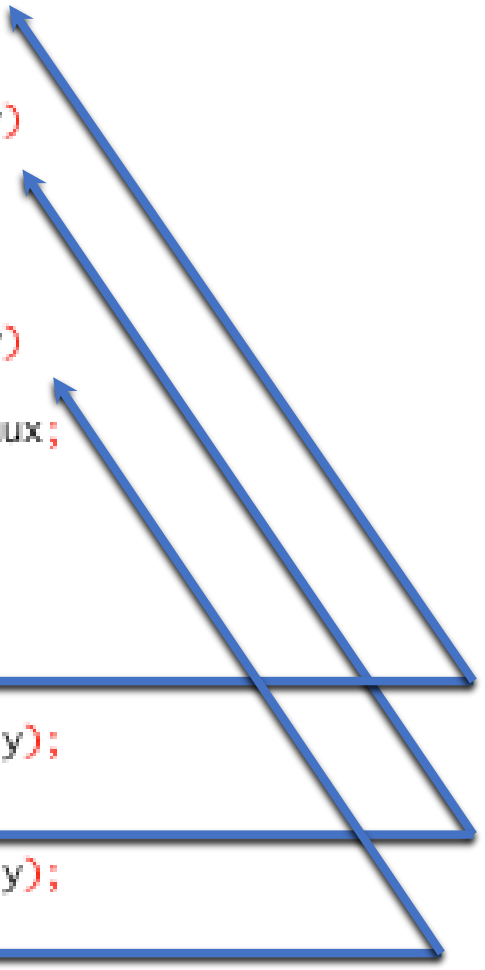
```cpp
#include <stdio.h>

void interschimba1(int x, int y)
{
    int aux = x; x = y; y = aux;
}

void interschimba2(int& x, int& y)
{
    int aux = x; x = y; y = aux;
}

void interschimba3(int* x, int* y)
{
    int aux = *x; *x = *y; *y = aux;
}

int main()
{
    int x=10, y =15;
    interschimba1(x,y);
    printf("x = %d, y = %d \n",x,y);
    x=10, y =15;
    interschimba2(x,y);
    printf("x = %d, y = %d \n",x,y);
    x=10, y =15;
    interschimba3(&x,&y);
    printf("x = %d, y = %d \n",x,y);
    return 0;
}
```



apel prin valoare

apel prin referință
numai în C++

apel prin valoare

interchimbare.cpp

```cpp
#include <stdio.h>

void interschimba1(int x, int y)
{
    int aux = x; x = y; y = aux;
}

void interschimba2(int& x, int& y)
{
    int aux = x; x = y; y = aux;
}

void interschimba3(int* x, int* y)
{
    int aux = *x; *x = *y; *y = aux;
}

int main()
{
    int x=10, y =15;
    interschimba1(x,y);
    printf("x = %d, y = %d \n",x,y);
    x=10, y =15;
    interschimba2(x,y);
    printf("x = %d, y = %d \n",x,y);
    x=10, y =15;
    interschimba3(&x,&y);
    printf("x = %d, y = %d \n",x,y);
    return 0;
}
```

```
x = 10, y = 15
x = 15, y = 10
x = 15, y = 10
```

apel prin valoare

apel prin referință
numai în C++

apel prin valoare

Transmiterea parametrilor către funcții

- în limbajul C transmiterea parametrilor se poate face doar prin **valoare (pass-by-value)**
  - ✓ o copie a argumentelor este trimisă funcției
  - ✓ **modificările în interiorul funcției nu afectează argumentele originale**

- pentru modificarea parametrilor actuali, funcţiei i se transmit nu valorile parametrilor actuali, ci **adresele lor (pass by pointer)**
  - ✓ Funcția face o copie a adresei, iar prin intermediul ei lucrează cu variabila "reală" (zona de memorie "reală").
  - ✓ Astfel **putem simula în C transmiterea prin referință cu ajutorul pointerilor.**

Transmiterea parametrilor către funcții

main.c

```c
#include <stdio.h>
#include <stdlib.h>

int f1(int a, int b)
{
    a++;
    b++;
    printf("In functia f1 avem a=%d,b=%d\n",a,b);
    return a+b;
}

int main(){
    int a = 5, b= 8;
    int c = f1(a,b);
    printf("In functia main avem a=%d,b=%d,c=%d\n",a,b,c);
    return 0;
}
```

Transmiterea parametrilor către funcții

main.c

```c
#include <stdio.h>
#include <stdlib.h>

int f1(int a, int b)
{
    a++;
    b++;
    printf("In functia f1 avem a=%d,b=%d\n",a,b);
    return a+b;
}

int main(){
    int a = 5, b= 8;
    int c = f1(a,b);
    printf("In functia main avem a=%d,b=%d,c=%d\n",a,b,c);
    return 0;
}
```

```
In functia f1 avem a=6,b=9
In functia main avem a=5,b=8,c=15
```

Transmiterea parametrilor către funcții

main.c

```c
#include <stdio.h>
#include <stdlib.h>

int f1(int a, int b)
{
    a++;
    b++;
    printf("In functia f1 avem a=%d,b=%d\n",a,b);
    return a+b;
}

int f2(int*a, int b)
{
    *a = *a + 1;
    b++;
    printf("In functia f2 avem *a=%d,b=%d\n",*a,b);
    return *a+b;

}
int main(){
    int a = 5, b= 8;
    int c = f2(&a,b);
    printf("In functia main avem a=%d,b=%d,c=%d\n",a,b,c);
    return 0;
}
```

Transmiterea parametrilor către funcții

main.c

```c
#include <stdio.h>
#include <stdlib.h>

int f1(int a, int b)
{
    a++;
    b++;
    printf("In functia f1 avem a=%d,b=%d\n",a,b);
    return a+b;
}

int f2(int*a, int b)
{
    *a = *a + 1;
    b++;
    printf("In functia f2 avem *a=%d,b=%d\n",*a,b);
    return *a+b;

}
int main(){
    int a = 5, b= 8;
    int c = f2(&a,b);
    printf("In functia main avem a=%d,b=%d,c=%d\n",a,b,c);
    return 0;
}
```

```
In functia f2 avem *a=6,b=9
In functia main avem a=6,b=8,c=15
```

Apelul funcției și revenirea din apel

Etapele principale ale apelului unei funcției și a revenirii din acesta în funcția de unde a fost apelată:

1) argumentele apelului sunt evaluate și trimise funcției
2) adresa de revenire este salvată pe stivă
3) controlul trece la funcția care este apelată
4) funcția apelată alocă pe **stivă** spațiu pentru variabilele locale și pentru cele temporare
5) se execută instrucțiunile din corpul funcției
6) dacă există valoare returnată, aceasta este pusă într-un loc sigur
7) spațiul alocat pe stivă este eliberat
8) utilizând adresa de revenire, controlul este transferat în funcția care a inițiat apelul

Funcții

1. Declarare și definire. Apel.
2. Metode de trasmitere a paramerilor.
3. **Pointeri la funcții**

Stiva în C

- **Stivă =** o structură internă utilizată la execuția programelor C pentru alocarea memoriei și manipularea variabilelor temporare
- pe stivă sunt alocate și memorate:
  - ✓ **variabilele locale** din cadrul funcțiilor
  - ✓ **parametrii** funcțiilor
  - ✓ **adresele de retur ale funcțiilor**
- dimensiunea implicită a stivei este redusă =>
  - ✓ în timpul execuției programele trebuie să nu depășească dimensiunea stivei
  - ✓ dimensiunea stivei poate fi modificată în prealabil din setările editorului de legături (*linker*)

Stiva în C – depășirea dimensiunii

Ambele programe eșuează în timpul execuției din cauza depășirii dimensiunii stivei

```
int    f1()
{
  int a[10000000]={0};
}

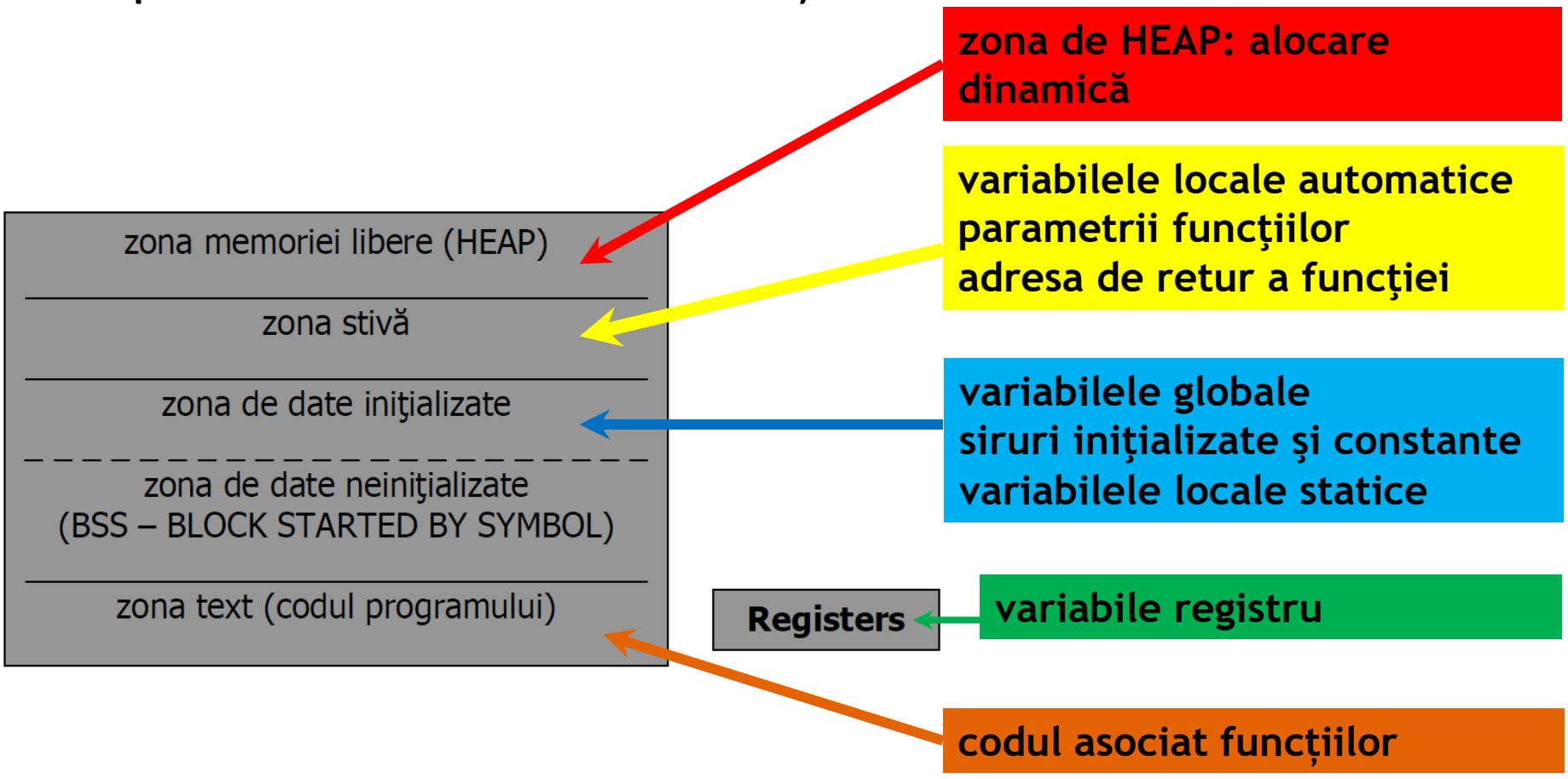
int    main()
{
     f1();
     return    0;
}
```

```
int    f2(int a,int  b)
{
     if (a<b)
          return 1+f2(a+1,b-1);
     else
          return 0;
}

int    main()
{
     printf("%d",f2(0,1000000)
     );
     return 0;
}
```

Pointeri la funcții

Pointer la o funcție = variabilă ce stochează adresa de început a codului asociat funcției



Schema memoriei la rularea unui program

hartaMemorie.c

```c
#include <stdio.h>
// variabile globale neinitializate
int g1,g2;
// variabile globale initializate
int g3=5, g4 = 7;
int g5, g6;

void f1() {
    int var1,var2;
    printf("In Stiva prin f1:\t\t %p %p\n",&var1,&var2);
}

void f2() {
    int var1,var2;
    printf("In Stiva prin f2:\t\t %p %p\n",&var1,&var2);
    f1();
}

int main() {
    //variabile locale
    int var1,var2;
    printf("In Stiva prin main:\t\t %p %p\n",&var1,&var2);
    f2();
    // variabile globale initializate + neinitializate
    printf("Variabile globale neinitializate:\t\t %p %p\n",&g1,&g2);
    printf("Variabile globale initializate:  \t\t %p %p\n",&g3,&g4);
    printf("Variabile globale neinitializate:\t\t %p %p\n",&g5,&g6);
    //cod
    printf("Text Data:\t\t\t\t\t %p %p \n\n",main,f1);
    return 0;
}
```

Schema memoriei la rularea unui program

hartaMemorie.c

```c
#include <stdio.h>
// variabile globale neinitializate
int g1,g2;
// variabile globale initializate
int g3=5, g4 = 7;
int g5, g6;

void f1() {
    int var1,var2;
    printf("In Stiva prin f1:\t\t %p %p\n",&var1,&var2);
}

void f2() { 0
    int var1,var2;
    printf("In Stiva prin f2:\t\t %p %p\n",&var1,&var2);
    f1();
}

int main() {
    //variabile locale
    int var1,var2;
    printf("In Stiva prin main:\t\t %p %p\n",&var1,&var2);
    f2();
    // variabile globale initializate + neinitializate
    printf("Variabile globale neinitializate:\t\t %p %p\n",&g1,&g2);
    printf("Variabile globale initializate:  \t\t %p %p\n",&g3,&g4);
    printf("Variabile globale neinitializate:\t\t %p %p\n",&g5,&g6);
    //cod
    printf("Text Data:\t\t\t\t\t %p %p \n\n",main,f1);
    return 0;
}
```

Numele unei funcții neînsoțit de o listă de argumente este adresa de început a codului funcției și este interpretat ca un pointer la funcția respectivă

Schema memoriei la rularea unui program

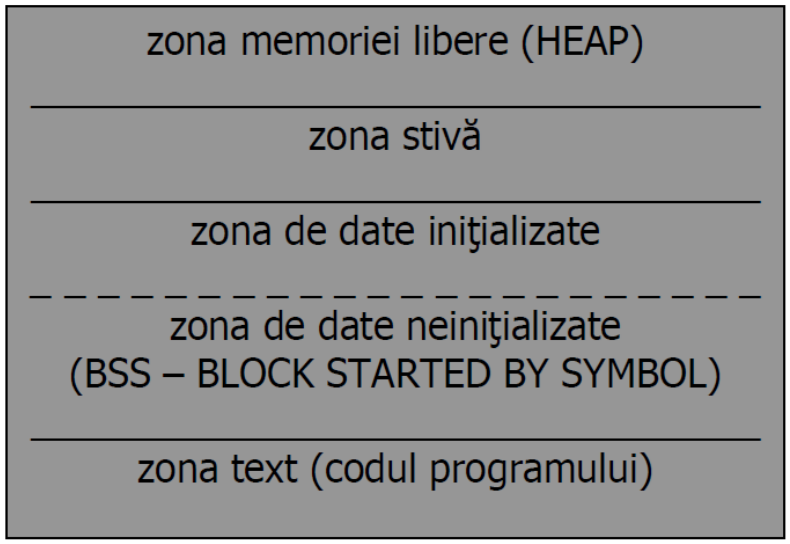
hartaMemorie.c

```c
#include <stdio.h>
// variabile globale neinitiali
int g1,g2;
// variabile globale initializa
int g3=5, g4 = 7;
int g5, g6;

void f1() {
    int var1,var2;
    printf("In Stiva prin f1:\t
}

void f2() {
    int var1,var2;
    printf("In Stiva
    f1();
}

int main() {
    //variabile loca
    int var1,var2;
    printf("In Stiva prin main:\t\t %p %p\n",&var1,&var2);
    f2();
    // variabile globale initializate + neinitializate
    printf("Variabile globale neinitializate:\t\t %p %p\n",&g1,&g2);
    printf("Variabile globale initializate:  \t\t %p %p\n",&g3,&g4);
    printf("Variabile globale neinitializate:\t\t %p %p\n",&g5,&g6);
    //cod
    printf("Text Data:\t\t\t\t\t %p %p \n\n",main,f1);
    return 0;
}
```




```
In Stiva prin main:                0x7fff5fbff95c 0x7fff5fbff958
In Stiva prin f2:                  0x7fff5fbff93c 0x7fff5fbff938
In Stiva prin f1:                  0x7fff5fbff91c 0x7fff5fbff918
Variabile globale neinitializate:                 0x100001070 0x100001074
Variabile globale initializate:                   0x100001068 0x10000106c
Variabile globale neinitializate:                 0x100001078 0x10000107c
Text Data:                                        0x100000d54 0x100000d04
```

Pointeri la funcții

- **pointer la o funcție** = variabilă ce stochează adresa de început a codului asociat funcției

Sintaxa: **tip (*nume_pointer_functie) (tipuri argumente)**

- ✓ **tip** = tipul de bază returnat de funcția spre care pointează nume_pointer_functie
- ✓ **nume_pointer_functie** = variabila de tip pointer la o funcție care poate lua ca valori adrese de memorie unde începe codul unei funcții

Observație: *trebuie să punem paranteză în definiție* ***(*pf)*** **,** *altfel definim o funcție care întoarce un pointer de date* ****pf***

Exemple:

```
void (*pf)(int)
int (*pf)(int, int)
double (*pf)(int, double*)
```

Pointeri la funcții

main.c

```c
#include <stdio.h>
#include <stdlib.h>

int suma(int a, int b)
{
    return a+b;
}


int main()
{
    int (*pf)(int,int);
    pf = &suma;
    int s1 = (*pf)(2,5);
    printf("s1 = %d\n",s1);
    pf = suma;
    int s2 = (*pf)(2,5);
    printf("s2 = %d\n",s2);
    return 0;
}
```

```
s1 = 7
s2 = 7
```

1. pentru a asigna unui pointer adresa unei functii, trebuie folosit **numele functiei fara paranteze.**
2. numele unei funcții este un pointer spre adresa sa de început din segmentul de cod: **f==&f**

Utilitatea pointerilor la funcții

- se folosesc în ***programarea generică***: realizăm apeluri de tip callback;
- o funcţie C transmisă printr-un pointer ca argument unei alte funcţii F se numeşte şi funcţie **“callback”**, pentru că ea va fi apelată “înapoi” de funcţia F

Exemplu: (funcția **qsort** din **stdlib.h**)

```
void qsort (void *adresa,
            int nr_elemente,
            int dimensiune_element,
            int (*cmp)(const void *, const void *)) ;
```

Utilitatea pointerilor la funcții

Exemplul 1: **Functii callback**

```
1  #include<stdio.h>
2  #include<stdlib.h>
3
4  double diferenta (int x, int y)
5      { return x - y;}
6
7  double media (int x, int y)
8      { return (x + y)/2.0;}
9
10 double calcul (int a, int b, double (*f)(int, int))
11     { return f(a, b);}
12
13 int main()
14 {
15     double x = calcul(10, 1, media);
16     double y = calcul(10, 1, diferenta);
17     printf("%g %g", x, y); // 5.5 9
18
19     return 0;
20 }
```

g sau G – general number

Utilitatea pointerilor la funcții

Ex. 2: Varianta cu selectie prin switch

```c
#include<stdio.h>
#include<stdlib.h>

int add(int a, int b) {return (a + b);}
int sub(int a, int b) {return (a - b);}
int mul(int a, int b) {return (a * b);}
int divide(int a, int b) {return (a / b);}

int main()
{
    int a = 10, b = 5, i, rez;
    printf("Introdu o valoare intre 0 si 3: ");
    scanf("%d", &i);

    switch(i)
    {
        case 0: rez = add(a,b); break;
        case 1: rez = sub(a,b); break;
        case 2: rez = mul(a,b); break;
        case 3: rez = divide(a,b); break;
        default: printf("Invalid!");
    }
    printf("%d", rez);

    return 0;
}
```

Utilitatea pointerilor la funcții

Ex. 2: Varianta cu selectie prin functii callback

```c
#include<stdio.h>
#include<stdlib.h>

int add(int a, int b) {return (a + b);}
int sub(int a, int b) {return (a - b);}
int mul(int a, int b) {return (a * b);}
int divide(int a, int b) {return (a / b);}

int (*op[4])(int a, int b) = {add, sub, mul, divide};

int main()
{
    int a = 10, b = 5, i, rez;
    printf("Introdu o valoare intre 0 si 3: ");
    scanf("%d", &i);

    rez = op[i](a, b);
    printf("%d", rez);

    return 0;
}
```

Utilitatea pointerilor la funcții

Exemplul 3: **funcția qsort din stdlib.h folosită pentru sortarea unui vector/tablou.**

Antetul funcției **qsort** este:

void qsort (void *adresa, int nr_elemente, int dimensiune_element, int (*cmp) (const void *, const void *))

- ✓ **adresa** = pointer la adresa primului element al tabloului ce urmeaza a fi sortat (pointer generic – nu are o aritmetică inclusă)
- ✓ **nr_elemente** = numarul de elemente al vectorului
- ✓ **dimensiune_element** = dimensiunea in octeți a fiecărui element al tabloului (char = 1 octet, int = 4 octeți, etc)
- ✓ **cmp / compare/ comparator– funcția de comparare a două elemente**

Utilitatea pointerilor la funcții

```
int cmp(const void *a, const void *b)
```



adresele a două elemente din tablou

- **cmp :**
  - ✓ o funcție generică comparator
  - ✓ **compară 2 elemente de orice tip**
  - ✓ întoarce:
    - ➢ un număr < 0, dacă vrem a la stânga lui b
    - ➢ un număr >0, dacă vrem a la dreapta lui b
    - ➢ 0, dacă nu contează

Utilitatea pointerilor la funcții

Exemplul 3: funcția qsort din stdlib.h folosită pentru sortarea unui vector/ tablou; exemplu de funcție **cmp** pentru sortarea unui vector de numere întregi:

cmp = compară 2 elemente a și b de orice tip și întoarce:

- ✓ un număr < 0, dacă vrem a la stânga lui b
- ✓ un număr >0, dacă vrem a la dreapta lui b
- ✓ 0, dacă nu contează

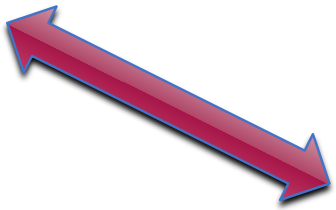
```
int cmp(const void *a, const void *b)
{
    int va, vb;
    va = *(int*)a;
    vb = *(int*)b;
    if(va < vb) return -1;
    if(va > vb) return 1;
    return 0;
}
```

Utilitatea pointerilor la funcții

Exemplul 3: **funcția qsort din stdlib.h folosită pentru sortarea unui vector/tablou.**

Exemplu de funcție cmp pentru sortarea unui vector de numere întregi:

```
int cmp(const void *a, const void *b)
{
    int va, vb;
    va = *(int*)a;
    vb = *(int*)b;
    if(va < vb) return -1;
    if(va > vb) return 1;
    return 0;
}
```



```
int cmp(const void *a, const void *b)
{
    return *(int*)a - *(int*)b;
}
```

Exemplul 3: **sortarea unui vector folosind funcția qsort**

```c
#include <stdio.h>
#include <stdlib.h>

int compare(const void *a, const void *b)
    { return (*(int*)a - *(int*)b);}

int comp(const void *a, const void *b)
    { return (*(float*)a - *(float*)b);}

int main()
{
  int v[] = { 5, 2, 9, 1, 7 };
  int size = sizeof(v) / sizeof(v[0]);
  qsort(v, size, sizeof(int), compare);

  for (int i = 0; i < size; i++)
    printf("%d ", v[i]);

  printf("\n");

  float w[] = { -5, 2.54, 9.1, -1, 7.2 };
  size = sizeof(w) / sizeof(w[0]);
  qsort(w, size, sizeof(float), comp);

  for (int i = 0; i < size; i++)
    printf("%.2f ", w[i]);

  return 0;
}
```

1 2 5 7 9
-5.00 -1.00 2.54 7.20 9.10

Cursul 6

1. Fișiere
2. Funcții. Pointeri la funcții

Cursul 7

1. Legătura dintre tablouri și pointeri
2. Aritmetica pointerilor
3. Alocare dinamică a memoriei
4. Clase de memorare